

Introduction to Embedded Systems

EMBEDED — Lecture 01

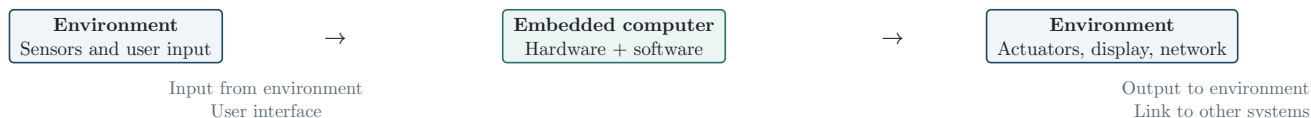


Fig. 1: Embedded computer between two environments: input from the environment and output to the environment; hardware and software work together to perform a dedicated function inside a larger system.

Embedded Systems: Definition

Definition Computing systems with **tightly coupled hardware and software integration**, designed to perform a **dedicated functionality**. The word “embedded” means built into (to be an integral part of) a larger system; the larger system is called the **embedding system**. An embedded system is an **application-specific computer system**.

- Many embedded systems must also respond within a defined time (**realtime constraints**).

Why Embed Computing?

- Lower cost and more performance**: application-specific and optimized system.
- Improve dependability**: one device fails but others continue.
- More functionality**: e.g., intelligence.
- Scalability**.

Bike Computer Example

- Wheel rotation becomes speed and distance on a local display.
- The design is shaped by **size, cost, weight, power, and energy**.
- A small **8-bit microcontroller (MCU)** can be enough.



Fig. 2: Bike computer: wheel rotation becomes speed and distance on a local display.

A Vehicle Is a Network of Computers

- Today’s high-end automobile may have **100 processors**:
 - many specialized controllers: a **4-bit microcontroller** checks the seat belt; microcontrollers run dashboard devices; a **16/32-bit microprocessor** controls the engine.
 - Low-end cars use **20+ processors**.
- Different **timing and safety needs**.

Microcontroller vs. Microprocessor

- Common to both**: a core to execute instructions; both devices fetch, decode, and execute instructions; both contain registers and an arithmetic logic unit (ALU).
- The main distinction lies in **what surrounds the processor core**.

CPU (Micro-processor)

A single processor core that supports at least instruction fetching, decoding, and executing. Used for general-purpose computing but depends on external memory and peripherals.

MCU (Micro-controller)

Typically has a single processor core; typically used for basic control purposes. Has memory blocks, Digital IOs, Analog IOs, and other basic peripherals. Integrates the processor, memory, and common peripherals on one chip.

Dedicated Hardware

- Custom logic implements a narrow, well-defined function:
 - high performance or energy efficiency;
 - updates are harder.
- The value of programmability**: the same hardware can perform different functions by changing only the software.

Alternatives

- Field-programmable gate arrays (FPGAs), custom logic, etc.
 - FPGAs offer **reconfigurable logic between fixed hardware and software**.

Why MCUs Fit Embedded Products

- Low development and manufacturing cost**: integrated peripherals reduce board complexity.
- Enough performance** for sensing and control.
- Low power consumption** and effective sleep modes.

Options for Building Embedded Systems

Dedicated Hardware

Discrete Logic	design cost low; unit cost mid; upgrades & bug fixes hard; size large; weight high; power ?; system speed very fast
ASIC	design cost high (\$500K/mask set); unit cost very low; upgrades hard; size tiny – 1 die; weight very low; power low; system speed extremely fast
Programmable logic – FPGA, PLD	design cost low; unit cost mid; upgrades easy; size small; weight low; power medium to high; system speed very fast

Software Running on Generic Hardware

Microprocessor + memory + peripherals	design cost low to mid; unit cost mid; upgrades easy; size small to med.; weight low to moderate; power medium; system speed moderate
Microcontroller (int. memory & peripherals)	design cost low; unit cost mid to low; upgrades easy; size small; weight low; power medium; system speed slow to moderate

Resource Constraints

- How much hardware do we need? How fast is the CPU? Memory size?
- How do we meet our deadlines? Faster hardware or cleverer software?
- How do we minimize power? Turn off unnecessary logic? Reduce memory accesses?

Correctness, Evolution, and Security

- Does it really work?** Is the specification correct? Does the implementation meet the specifications? How do we test for real-time characteristics? How do we test on real data?
- How do we design for upgradeability?** How to add features by changing software and hardware?
- Is it secure?** How are the dangerous acts prevented?

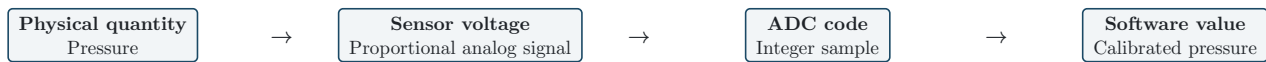


Fig. 3: Analog sensing path: physical quantity, sensor voltage, ADC code, software value.

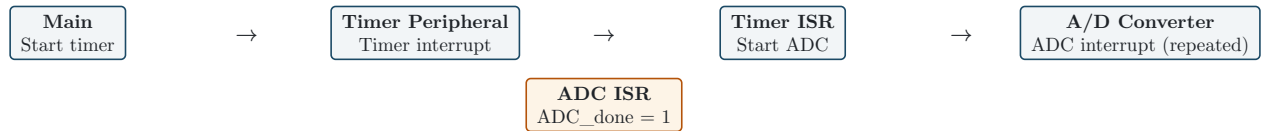


Fig. 5: Interrupt-driven execution: the main program runs while a timer peripheral and an A/D converter peripheral drive the Timer ISR and the ADC ISR through interrupts.

Why Embedded Testing Is Difficult

- **Complex testing:** we cannot separate the testing of an embedded computer from the machine in which it is embedded (how to test a washing machine software?).
- **Limited observability and controllability:** difficult to see what is going on; no keyboards and screens, need to watch the values of electrical signals.
- **Restricted development environments:** much more limited than those available for PCs.

Core Properties

- **Interfacing with the environment:** analog signals from sensors, e.g., a voltage value that represents a physical value.
- **Concurrent and reactive behaviours:** respond to events timely (real-time constraints on responses); must perform multiple separate activities concurrently (e.g., several sensors).
- **Fault handling:** operate independently for long periods and handle likely faults without crashing.
- **Diagnostics:** help developers determine problems quickly.

Example: Analog Sensing Path

- A sensor detects pressure and generates a **proportional output voltage** V_{sensor} .
- An **Analog to Digital Converter (ADC)** generates a proportional digital integer (code) based on V_{sensor} and V_{ref} .

From ADC Code to Voltage

- The ADC converts a **voltage range into integer codes**.
 - Reference voltage and ADC resolution define the **conversion scale**.
 - **Clipping** occurs when the input exceeds the supported range.
- The MCU reads these integer codes; the MCU software should convert the measured integer code back into the voltage reading.

Formula

$$V_{\text{sensor}} = \text{ADC_code} \times V_{\text{ref}} / \text{ADC_max}$$

```
// Your Software
ADC_Code = adc_read();
V_sensor = ADC_code * V_ref / ADC_MASK;
```

From Voltage to Pressure

- The **sensor model** converts voltage into pressure.
 - Calibration constants, units, and environmental assumptions must remain **explicit**.

```
// Your Software
ADC_Code = adc_read();
V_sensor = ADC_code * V_ref / ADC_MASK;
Pressure_kPa = 250 * (V_sensor / V_supply + 0.04);
```

From Pressure to Depth

- A **second model** converts pressure difference into depth.
 - Calibration constants, units, and environmental assumptions must remain **explicit**.
 - Each conversion adds **uncertainty** that software must manage.

Formula

$$\text{depth} = f(\text{pressure} - \text{atmospheric pressure})$$

```
// Your Software
ADC_Code = adc_read();
V_sensor = ADC_code * V_ref / ADC_MASK;
Pressure_kPa = 250 * (V_sensor / V_supply + 0.04);
Depth_ft = 33 * (Pressure_kPa - Atmos_Press_kPa) / 101.3;
```

Concurrency Through Peripherals

- Things are happening **concurrently** while the CPU is executing instructions: peripherals continue working while the CPU executes application code.
- Peripherals use **interrupts** to notify the CPU about events:
 - an interrupt transfers control to a short **service routine**;
 - the routine records the event or moves data;
 - the main program resumes after the urgent work is complete.

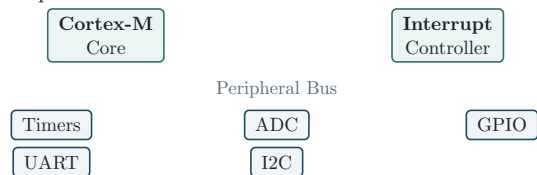


Fig. 4: Concurrency through peripherals: the Cortex-M core and the interrupt controller are connected to peripherals on the peripheral bus.

Interrupt-Driven Execution

Embedded Software

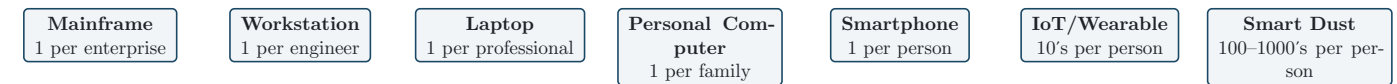
- **Programming language:** programmed in **C** rather than **Java** (smaller and faster code, so less expensive MCU); some performance-critical code may be in **assembly language**.
- **Operating system:** typically **no OS**, instead a simple scheduler, or even just interrupts + main code (foreground/background system); if an OS is used, it is likely to be an **embedded RTOS**.

Hardware and Software Co-design Model

- Commonly both the hardware and the software for an embedded system are developed **in parallel**.
- If performance is needed: take advantage of special hardware features.
- Push things to the **software layer** if functionality can be achieved in software: reduces overall hardware complexity and cost.

Functional vs. Non-functional Requirements

Functional	Output as a function of input.
Non-functional	Time required to compute output; size, weight (mobile and portable); power consumption (battery capacity); reliability.
• Functionality is a must but embedded applications should meet non-functional requirements as well. • Performance goal: the program must meet its deadline. • Deadline: the time at which a computation must be finished. • Example: TV broadcast involves digital signal processing and cannot tolerate delays. • If the program does not produce the required output by the deadline, the system does not work properly even if the output it eventually produces is functionally correct. • A digital signal processor (DSP) has specialized arithmetic units to perform complex calculations extremely fast in real time.	



“Roughly every decade a new, lower priced computer class forms based on a new programming platform, network, and interface resulting in new usage and the establishment of a new industry.” — Bell et al., *Computer*, 1972, ACM, 2008

Fig. 7: Growth of computing classes, from mainframe to smart dust.

The Internet of Things (IoT)

- Embedded devices **exchange data** and **coordinate with software services** across the physical world.

IoT Device Examples

- Different products share one pattern: **sensing, local computation, communication, and action.**

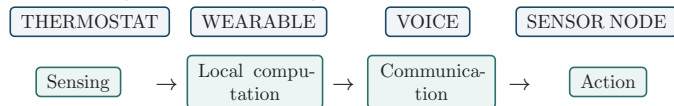


Fig. 6: IoT device examples sharing one pattern: sensing, local computation, communication, action.

Why Connect Embedded Devices?

- Collect measurements beyond the reach of wired infrastructure.
- Manage devices and update behavior remotely.
- Coordinate devices that share a physical process.
- Combine local sensing with remote storage and analysis.
- Expose new **security, privacy, and reliability risks**.

Wireless Networking

- Networking is a **critical component** of an IoT system: allows a much wider range of sensor applications.
- Why IoT? Items can have more functionality and become more intelligent; items can be managed more easily; more information becomes available.

Trillion Embedded Devices Will Be Deployed Soon

- Graph of log(people per computer) over time (1950–2020), with computer classes: Mainframe (1 per enterprise), Workstation (1 per engineer), Laptop (1 per professional), Personal Computer (1 per family), Smartphone (1 per person), IoT/Wearable (10's per person), Smart Dust (100–1000's per person).

Cyber-physical Systems

- Combines **physical devices** with **computers that control the device**.
- The embedded computer is the **cyber part**:
 - replaces mechanical controllers: more accurate and more sophisticated control;
 - monitors and controls the physical processes with **feedback loops**.

Summary

- An embedded system is built for a **specific application**: hardware and software components.
- Embedded systems pose many **design challenges**: functional and non-functional requirements (design time, deadlines, power, cost, etc.).
- In **real-time systems**, timing correctness is just as important as functional or logical correctness.